

Università degli Studi di Torino
Corso di Laurea Magistrale in Informatica

Intelligenza Artificiale
Progetto Clingo

Pianificazione di Torneo Pokémon
con Answer Set Programming
Sistema Svizzero – Relazione Tecnica Dettagliata

Autore:
Alessandro Livero

Anno Accademico 2025/2026

28 giugno 2026

Indice

1	Introduzione e Contesto	2
1.1	Il Sistema Svizzero	2
1.2	Answer Set Programming (ASP)	2
1.3	File del Progetto	2
2	Programma torneo.lp – Analisi Dettagliata	2
2.1	Sezione 1: Normalizzazione e Ordinamento	2
2.1.1	played_already/2 – Simmetria degli Incontri	2
2.1.2	pid/2 – Indice Canonico	3
2.2	Sezione 2: Generazione (Choice Rules)	3
2.2.1	match/2 – Generazione degli Incontri	3
2.2.2	bye/1 – Generazione del BYE	4
2.3	Sezione 3: Vincoli Hard (Integrity Constraints)	4
2.3.1	Helper in_match/1	4
2.3.2	Vincolo: Ogni Giocatore Deve Fare Qualcosa	4
2.3.3	Vincolo: Un Giocatore in un Solo Match	4
2.3.4	Vincolo: Un Solo BYE per Turno	5
2.3.5	Vincolo di Parità: BYE iff Numero Dispari	5
2.4	Sezione 4: Ottimizzazione	5
2.4.1	score_diff/3 – Differenza di Punteggio	5
2.4.2	#minimize – Funzione di Ottimizzazione	5
2.5	Sezione 5: Output	6
3	Istanze: turno1.lp e turno2.lp	6
3.1	Turno 1 – 5 Giocatori, Tutti a 0 Punti	6
3.2	Turno 2 – 8 Giocatori, Punteggi Differenziati	7
4	Concetti ASP Fondamentali Usati nel Progetto	8
4.1	Riepilogo dei Costrutti	8
4.2	Ciclo di Vita di un Answer Set	8
4.3	Negazione per Fallimento vs Negazione Classica	9
5	Architettura Complessiva e Flusso	9
5.1	Separazione Programma / Istanza	9
6	Proprietà Formali e Correttezza	10
6.1	Completezza: Ogni Abbinamento Valido è Generato	10
6.2	Unicità della Forma Canonica	10
6.3	Ottimalità dell’Abbinamento	10

1 Introduzione e Contesto

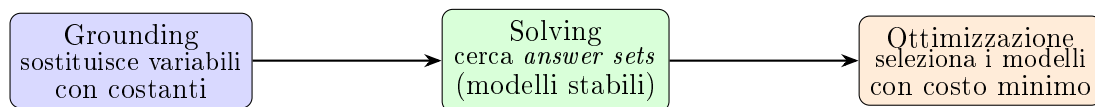
1.1 Il Sistema Svizzero

Il **sistema svizzero** è un formato di torneo usato in scacchi, Pokémon competitivo, Magic: The Gathering e altri giochi. A ogni turno i giocatori vengono abbinati secondo queste regole:

1. Si abbinano giocatori con punteggio simile (*spareggio*).
2. Due giocatori non si affrontano mai due volte.
3. Se il numero di giocatori è dispari, uno riceve il **BYE** (vittoria automatica senza giocare); nessuno lo può ricevere due volte.

1.2 Answer Set Programming (ASP)

L'ASP è un paradigma di programmazione logica orientato alla soluzione di problemi combinatori. Il **grounding** + **solving** di Clingo funziona in due fasi:



Un **answer set** è un insieme di atomi ground che soddisfano tutte le regole e i vincoli del programma. Con più answer set, Clingo può restituire quello con il valore di ottimizzazione minimo (`-opt-mode=optN`).

1.3 File del Progetto

File	Tipo	Contenuto
torneo.lp	Programma ASP	Logica generica del torneo (invariante)
turno1.lp	Istanza	5 giocatori, tutti a 0 punti, turno 1
turno2.lp	Istanza	8 giocatori, punteggi aggiornati, turno 2

Uso da terminale:

```
clingo torneo.lp turno1.lp 1 --opt-mode=optN
clingo torneo.lp turno2.lp 1 --opt-mode=optN
```

Il parametro 1 chiede la prima soluzione ottimale; 0 restituirebbe tutte le soluzioni ottimali.

2 Programma torneo.lp – Analisi Dettagliata

2.1 Sezione 1: Normalizzazione e Ordinamento

2.1.1 played_already/2 – Simmetria degli Incontri

```
1 played_already(P1, P2) :- played(P1, P2).
2 played_already(P1, P2) :- played(P2, P1).
```

Il fatto `played(p1, p2)` nell'istanza è **direzionale** (per comodità di scrittura), ma semanticamente un incontro è simmetrico: se Lorenzo ha giocato contro Emma, Emma ha giocato contro Lorenzo.

`played_already/2` chiude questa relazione rispetto alla simmetria: entrambi gli ordini sono derivati.

2.1.2 pid/2 – Indice Canonico

```
1 pid(P, I) :- player(P),
2             I = #count{ Q : player(Q), Q != P, Q < P }.
```

Aggregato #count

`#count{ Q : Cond }` conta il numero di valori distinti di `Q` che soddisfano `Cond`. Il risultato è un intero ≥ 0 .

Cosa calcola `pid`:

$$\text{pid}(P, I) \iff I = |\{Q \in \text{player} \mid Q \neq P \wedge Q < P\}|$$

Poiché Clingo ordina i simboli lessicograficamente (come stringhe), `pid` assegna a ogni giocatore un indice intero univoco basato sull'ordine lessicografico dei nomi. Con i giocatori {diego, emma, giada, lorenzo, simone}:

Giocatore	pid
diego	0
emma	1
giada	2
lorenzo	3
simone	4

Perché serve? Per garantire che ogni coppia $(P1, P2)$ sia considerata *una sola volta* nella generazione: solo la forma con `pid(P1) < pid(P2)` viene usata. Senza questo ordinamento, Clingo genererebbe sia `match(emma, diego)` sia `match(diego, emma)`.

2.2 Sezione 2: Generazione (Choice Rules)

2.2.1 match/2 – Generazione degli Incontri

```
1 { match(P1, P2) } :-
2   player(P1), player(P2),
3   pid(P1, I1), pid(P2, I2), I1 < I2,
4   not played_already(P1, P2).
```

Choice Rule { h } :- Body

Una **choice rule** (regola di scelta) in ASP indica che l'atomo `h` può essere incluso nell'answer set se il corpo è soddisfatto, ma non è obbligatorio. Clingo esplora en-

trambe le possibilità (incluso / non incluso) e mantiene solo i modelli che soddisfano tutti i vincoli.

Condizioni per la generazione di match(P1, P2):

1. Entrambi sono giocatori registrati.
2. $\text{pid}(P1) < \text{pid}(P2)$: forma canonica (evita duplicati).
3. $\text{not played_already}(P1, P2)$: non si sono già affrontati.

2.2.2 bye/1 – Generazione del BYE

```
1 { bye(P) } :- player(P), not had_bye(P).
```

Ogni giocatore che non ha mai ricevuto il BYE *può* riceverlo in questo turno. I vincoli hard successivi limitano a zero o uno i BYE effettivi.

2.3 Sezione 3: Vincoli Hard (Integrity Constraints)

Integrity Constraint :- Body

Un vincolo di integrità :- Body elimina dall'insieme degli answer set qualsiasi modello in cui Body è vero. In sostanza: “è **vietato** che Body sia soddisfatto”.

2.3.1 Helper in_match/1

```
1 in_match(P) :- match(P, _).  
2 in_match(P) :- match(_, P).
```

Deriva che P è impegnato in un match, indipendentemente dal ruolo (primo o secondo argomento). Questo semplifica i vincoli successivi.

2.3.2 Vincolo: Ogni Giocatore Deve Fare Qualcosa

```
1 :- player(P), not in_match(P), not bye(P).  
2 :- player(P), in_match(P), bye(P).
```

Prima riga: è vietato che un giocatore non sia né in un match né abbia il BYE. Ogni giocatore deve partecipare.

Seconda riga: è vietato che un giocatore sia contemporaneamente in un match e abbia il BYE. Le due alternative sono mutualmente esclusive.

2.3.3 Vincolo: Un Giocatore in un Solo Match

```
1 :- match(P, Q1), match(P, Q2), Q1 != Q2.  
2 :- match(P1, Q), match(P2, Q), P1 != P2.  
3 :- match(P, _), match(_, P).
```

Tre vincoli che coprono tutti i casi di conflitto:

Vincolo	Proibisce	Esempio vietato
1	P appare come P1 in due match diversi	<code>match(emma,diego), match(emma,giada)</code>
2	Q appare come P2 in due match diversi	<code>match(diego,emma), match(giada,emma)</code>
3	P appare in entrambi i ruoli	<code>match(emma,diego), match(giada,emma)</code>

2.3.4 Vincolo: Un Solo BYE per Turno

```
1 :- 2 <= #count{ P : bye(P) }.
```

Vietato che due o più giocatori abbiano il BYE nello stesso turno. La sintassi $N \leq \text{Agg}$ è equivalente a $\text{Agg} \geq N$.

2.3.5 Vincolo di Parità: BYE iff Numero Dispari

```
1 player_count(N) :- N = #count{ P : player(P) }.
2 :- player_count(N), 0 == N \ 2, bye(_).
3 :- player_count(N), 1 == N \ 2, not bye(_).
```

`player_count/1`: deriva il numero totale di giocatori usando `#count`.

$N \setminus 2$: operatore modulo in Clingo (resto della divisione intera).

- Se $N \bmod 2 = 0$ (numero pari): vietato che esista un BYE.
- Se $N \bmod 2 = 1$ (numero dispari): vietato che non esista un BYE.

Il BYE è obbligatorio esattamente quando il numero di giocatori è dispari.

2.4 Sezione 4: Ottimizzazione

2.4.1 `score_diff/3` – Differenza di Punteggio

```
1 score_diff(P1, P2, D) :-
2     match(P1, P2), score(P1, S1), score(P2, S2),
3     S1 >= S2, D = S1 - S2.
4
5 score_diff(P1, P2, D) :-
6     match(P1, P2), score(P1, S1), score(P2, S2),
7     S2 > S1, D = S2 - S1.
```

Calcola la **differenza assoluta** di punteggio tra i due giocatori di ogni match. Le due clausole coprono i due casi ($S1 \geq S2$ e $S2 > S1$) per evitare valori negativi. Si noti che in ASP le espressioni aritmetiche come $D = S1 - S2$ sono valutate solo se $S1$ e $S2$ sono già ground (istanziate).

2.4.2 `#minimize` – Funzione di Ottimizzazione

```
1 #minimize { D, P1, P2 : score_diff(P1, P2, D) }.
```

```
#minimize { peso, chiave1, chiave2, ... : Cond }
```

Minimizza la **somma** dei pesi di tutti gli elementi dell'insieme che soddisfano la condizione. La tupla (peso, chiave1, chiave2, ...) identifica univocamente ogni elemento (le chiavi evitano conteggi doppi per elementi con lo stesso peso ma identità diverse).

In questo caso: si minimizza $\sum_{(P1,P2) \in \text{match}} |S_{P1} - S_{P2}|$.

Effetto pratico: abbinamenti con giocatori a pari punteggio hanno differenza 0 (contributo 0 alla somma) e sono preferiti; abbinamenti tra giocatori a punteggio molto diverso sono penalizzati.

Questo implementa il principio base del sistema svizzero: gli abbinamenti dello stesso “scoreboard” giocano tra loro.

2.5 Sezione 5: Output

```
1 #show match/2.  
2 #show bye/1.
```

```
#show predicato/arità
```

Per default, Clingo mostra *tutti* gli atomi dell'answer set. La direttiva `#show` limita l'output ai soli predicati elencati. Senza `#show`, verrebbero visualizzati anche `pid`, `in_match`, `played_already`, ecc.

3 Istanze: turno1.lp e turno2.lp

3.1 Turno 1 – 5 Giocatori, Tutti a 0 Punti

```
1 player(lorenzo). player(emma). player(simone).  
2 player(diego).   player(giada).  
3  
4 score(lorenzo, 0). score(emma, 0). score(simone, 0).  
5 score(diego, 0).   score(giada, 0).  
6 % Nessun played/2, nessun had_bye/1
```

Situazione: torneo appena iniziato.

- 5 giocatori (dispari): un BYE obbligatorio.
- Tutti a 0 punti: la differenza di punteggio è 0 per qualsiasi abbinamento.
- Nessun incontro già avvenuto: tutti gli abbinamenti sono validi.
- Nessun BYE precedente: chiunque può riceverlo.

Con tutti i punteggi uguali, l'ottimizzatore minimizza una somma di zeri: *qualsiasi* abbinamento è ottimo. Clingo restituirà una soluzione arbitraria tra quelle valide (dipende dall'ordine di esplorazione).

Output reale – clingo torneo.lp turno1.lp 1 -opt-mode=optN

```
Answer: 1
bye(simone) match(giada,lorenzo) match(diego,emma)
Optimization: 0
OPTIMUM FOUND
```

La funzione obiettivo vale **0**, come previsto: tutti i giocatori sono a pari punteggio, quindi ogni possibile combinazione di 2 match + 1 bye è equivalentemente ottima (Clingo ne trova più di una con lo stesso costo durante la prova di ottimalità).

3.2 Turno 2 – 8 Giocatori, Punteggi Differenziati

```
1 % Vecchi giocatori con punteggi aggiornati
2 player(lorenzo). score(lorenzo, 2).
3 player(simone). score(simone, 2).
4 player(giada). score(giada, 2).
5 player(emma). score(emma, 0).
6 player(diego). score(diego, 0).
7 % Nuovi iscritti (tutti a 0)
8 player(jacopo). score(jacopo, 0).
9 player(thomas). score(thomas, 0).
10 player(desiree). score(desiree, 0).
11
12 % Incontri del turno 1
13 played(emma, lorenzo).
14 played(diego, simone).
15 % had_bye: giada non puo' ricevere di nuovo il BYE
16 had_bye(giada).
```

Analisi:

- 8 giocatori (pari): nessun BYE questo turno.
- I giocatori si raggruppano per punteggio: $\{2\} = \{\text{lorenzo, simone, giada}\}$ e $\{0\} = \{\text{emma, diego, jacopo, thomas, desiree}\}$.
- `played(emma, lorenzo)` e `played(diego, simone)`: questi abbinamenti non possono ripetersi.
- L'ottimizzatore cercherà di abbinare i "2" tra loro e i "0" tra loro.

Output reale – clingo torneo.lp turno2.lp 1 -opt-mode=optN

```
Answer: 1
match(giada,lorenzo) match(desiree,simone)
match(diego,jacopo) match(emma,thomas)
Optimization: 2
OPTIMUM FOUND
```

Come previsto, **giada** e **lorenzo** (entrambi a 2 punti) vengono abbinati tra loro a costo 0. Poiché i giocatori a 2 punti sono 3 (numero dispari nel loro gruppo), **simone** resta senza un pari-punteggio disponibile e viene abbinato a **desiree** (0 punti): questo “*across-group*” match contribuisce $|2 - 0| = 2$ alla somma, che è

esattamente il valore minimo raggiungibile. Gli altri giocatori a 0 punti (**diego**, **emma**, **jacopo**, **thomas**) si abbinano liberamente tra loro a costo 0, rispettando i vincoli su `played_already`.

4 Concetti ASP Fondamentali Usati nel Progetto

4.1 Riepilogo dei Costrutti

Costrutto	Sintassi	Semantica
Fatto	<code>p(a).</code>	<code>p(a)</code> è sempre vero
Regola normale	<code>h :- b1, b2.</code>	<code>h</code> è vero se <code>b1</code> e <code>b2</code> sono veri
Negazione per fallimento	<code>not p</code>	vero se <code>p</code> non è nell'answer set
Choice rule	<code>{h} :- b.</code>	<code>h</code> può o non può essere incluso
Vincolo hard	<code>:- b.</code>	elimina i modelli dove <code>b</code> è vero
Aggregato count	<code>#count{x : p(x)}</code>	numero di <code>x</code> con <code>p(x)</code>
Modulo	<code>N \ M</code>	resto di <code>N</code> diviso <code>M</code>
Minimizzazione	<code>#minimize{w,k : p}.</code>	minimizza la somma dei pesi
Direttiva show	<code>#show p/n.</code>	mostra solo il predicato <code>p</code>

4.2 Ciclo di Vita di un Answer Set

1. **Grounding**: ogni variabile è sostituita con tutte le costanti compatibili. Le regole con variabili diventano insiemi di regole ground.
2. **Solving**: il solver esplora esponenzialmente le possibili combinazioni di atomi. Per ogni combinazione verifica:
 - (a) Tutte le regole normali sono soddisfatte (le teste sono derivate quando i corpi sono veri).
 - (b) Tutti i vincoli hard sono soddisfatti (nessun corpo vietato è vero).
 - (c) Il modello è *minimo*: nessun sottoinsieme è ancora un modello stabile.
3. **Ottimizzazione**: tra tutti gli answer set validi, Clingo seleziona quelli con il valore della funzione obiettivo minimo.

4.3 Negazione per Fallimento vs Negazione Classica

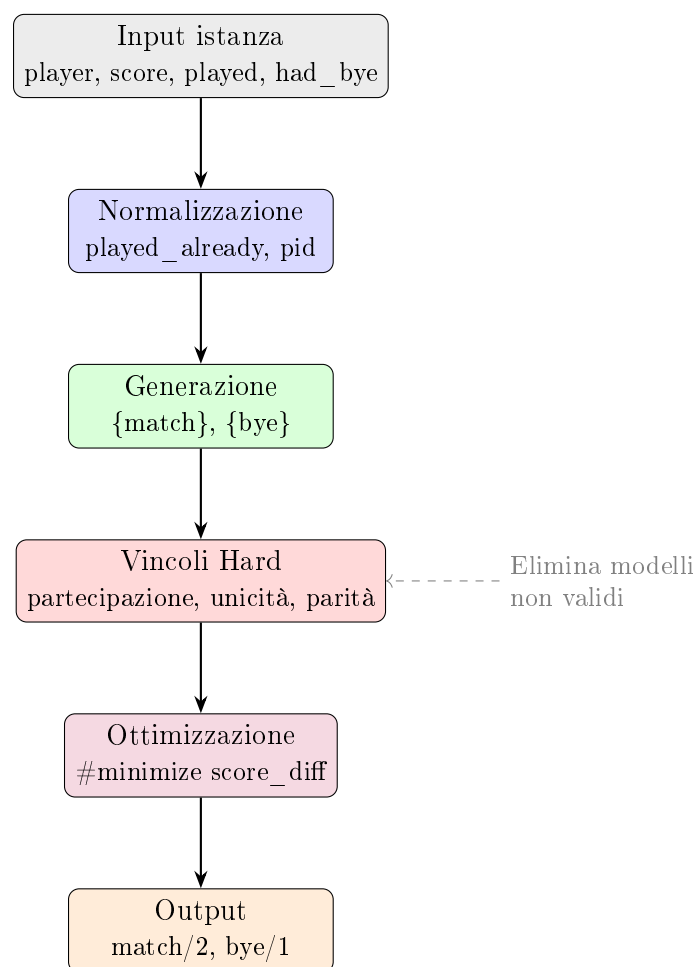
Differenza critica

Negazione classica ($\neg p$): afferma esplicitamente la falsità di p . Richiede che il programma contenga $\neg p$ come fatto o conclusione.

Negazione per fallimento ($\text{not } p$): vera se p *non appare* nell'answer set corrente (assunzione del mondo chiuso).

Nel progetto si usa solo **not**, appropriato perché: se non è noto che un giocatore abbia avuto il BYE (`had_bye`), si assume che non lo abbia avuto.

5 Architettura Complessiva e Flusso



5.1 Separazione Programma / Istanza

La separazione tra `torneo.lp` (logica) e i file di istanza (`turno1.lp`, `turno2.lp`) è il punto di forza del paradigma ASP:

- Per aggiungere un nuovo turno basta creare un nuovo file istanza.
- La logica del torneo non va mai modificata.
- Diversi tornei (con regole diverse) potrebbero condividere le istanze.

6 Proprietà Formali e Correttezza

6.1 Completezza: Ogni Abbinamento Valido è Generato

La choice rule `{match(P1,P2)}` genera *ogni* coppia di giocatori che non si siano già affrontati (nella forma canonica). I vincoli hard poi filtrano le combinazioni non valide. Il solver esplora tutte le possibilità: nessun abbinamento valido viene perduto.

6.2 Unicità della Forma Canonica

L'uso di `pid(P1, I1)`, `pid(P2, I2)`, `I1 < I2` garantisce che ogni coppia $\{P1, P2\}$ appaia esattamente una volta nella generazione. Senza questo ordinamento, il grounding produrrebbe sia `match(A,B)` che `match(B,A)` come atomi indipendenti, e i vincoli dovrebbero essere molto più complessi per eliminarli.

6.3 Ottimalità dell'Abbinamento

Il `#minimize` non garantisce che l'abbinamento sia “perfetto” nel senso matematico (problema di accoppiamento pesato su grafo, NP-completo in generale), ma trova il **minimo globale** della somma delle differenze, che è la definizione standard di abbinamento ottimale nel sistema svizzero.